

An Encoding-Aware Hitting-Set to Set-Cover Adapter

Reviewed research manuscript; awaiting expert assessment
23 September 2026

Abstract

Incidence duality sends hitting sets to set covers while preserving cardinality. We give a deterministic JSON instance map and output decoder for exact minimum hitting set and exact minimum set cover, including infeasible instances and every optimal tie. The map indexes target sets by elements that actually occur in the written source family, so even a decimal-encoded universe of enormous numeric size produces an instance polynomial in the input length. A direct proof establishes the recovery contract and polynomial bit complexity. Independent finite checks exercise 120 prepared sources, 689 exhaustively enumerated small sources, and sparse sources with 5,001-digit universe sizes. The duality itself is established prior work; the contribution here is its complete executable endpoint and encoding-aware bound.

1. Introduction

Minimum hitting set and minimum set cover express the same covering incidence relation with opposite choices. Their duality is standard: Cygan et al. state the one-to-one correspondence between hitting sets and set covers of the dual family in Section 4.1, immediately before Observation 4.1 [1]. A public reduction description also gives the direct instance map, index recovery, and corner cases [2]. These sources settle the combinatorial idea.

The executable endpoint has an encoding detail. If a source universe is represented by a decimal size, its numeric value can be much larger than the written input. Creating one target set for every possible source element can therefore take exponential time in input length. We index only elements present in a source member. Minimum hitting sets never use the omitted elements.

This is our main result: the active-element incidence map and its decoder form a deterministic polynomial-time reduction between the exact optimization endpoints. They preserve infeasibility and recover an optimum from every valid target optimum. The proof below covers empty and repeated members and gives bit-size bounds for the actual JSON encoding.

2. Instances and outputs

Let the source universe be $U = \{i : 0 \leq i < n\}$, with $n \geq 0$, and let $C = (C_j)_{0 \leq j < m}$ be an indexed family of subsets of U . A source hitting set $H \subseteq U$ meets every member C_j . A valid source output is a minimum-cardinality such H , or the distinguished answer `N0-SOLUTION` when none exists. For the target, a valid output selects a minimum-cardinality indexed subfamily covering its universe, or the same distinguished answer when no cover exists. Repeated and empty sets are legal in both families.

The input encoding is a JSON object with a decimal `universe_size` and an `indexed_sets` array of arrays of distinct in-range integers. Output JSON is an object with a `selected` array of distinct indices, or the string `N0-SOLUTION`. Decimal universe size is a compact field; the encoding does not list all n elements. Empty arrays and an empty universe are legal. The precise executable contract is recorded alongside the implementation.

3. Construction and recovery

Let $A = \bigcup_{j=0}^{m-1} C_j$ be the active source set, with the empty union understood as the empty set, and list its distinct elements in increasing order as $(a_i)_{0 \leq i < q}$. The target universe has one element per source member. Its target set at index i records every member containing a_i :

$$V = \{0, \dots, m-1\}, \quad D_i = \{j \in V : a_i \in C_j\} \quad (0 \leq i < q). \quad (1)$$

The small instance in Table 1 has columns for active source elements and target-set indices, and rows for source members and target-universe elements. Each entry records the same incidence before and after the change of viewpoint.

source member	$a_0 = 1$	$a_1 = 2$	$a_2 = 4$
$C_0 = \{1, 4\}$	1	0	1
$C_1 = \{4\}$	0	0	1
$C_2 = \{1, 2\}$	1	1	0

Table 1: Exact incidence matrix for $U = \{0, \dots, 5\}$ with three source members. Columns become target sets $D_0 = \{0, 2\}$, $D_1 = \{2\}$, and $D_2 = \{0, 1\}$. The unused elements 0, 3, and 5 create no target sets.

For a selected target index set I , recovery reconstructs the ordered active list from the source input and returns $H_I = \{a_i : i \in I\}$. It returns NO-SOLUTION unchanged. Recovery reads the source again; it has no dependence on the forward process’s memory. Both maps are implemented by the two command modes of `algorithm.py`.

4. Correctness

Theorem 1. For every legal source instance x , the map in Equation 1 produces a legal set-cover instance. For every valid target output y , the decoder returns a valid source output. In particular, it recovers a globally minimum hitting set from every globally minimum cover, and returns NO-SOLUTION exactly for infeasible sources.

Proof. Every D_i is a subset of V , so the constructed instance is legal. For any target index set I and any source-member index j , definition Equation 1 gives

$$j \in \bigcup_{i \in I} D_i \iff \exists i \in I : a_i \in C_j \iff H_I \cap C_j \neq \emptyset. \quad (2)$$

Thus I covers V exactly when H_I hits all members. The active elements are distinct, so $|I| = |H_I|$. Conversely, any source hitting set H remains feasible after deleting elements outside A , because every C_j lies in A . An optimum H cannot contain an element outside A , since deleting it would reduce cardinality. Feasibility and minimum cardinality therefore agree on the two sides. Equation Equation 2 sends every target optimum to a source optimum, including every tie. It also shows that one side is infeasible exactly when the other is; the distinguished answers correspond.

An empty source member leaves its target element uncovered. An empty source family yields $A = \emptyset$ and $V = \emptyset$, so the empty selection is optimal. Repeated members remain separately indexed target elements; unused source elements are absent from A . These cases all follow from Equation 2. $\square$

The same equivalence gives a conditional threshold statement. If a separate feasibility interface attaches the same integer threshold k to both instances, a hitting set of size at most k exists exactly when a cover of size at most k exists. In that separately specified interface, NO-SOLUTION is the answer exactly when no such selection exists. The executable contract here defines the exact optimization endpoint; it does not define a threshold field or threshold output mode.

5. Bit complexity

Let L be the source JSON length, $r = \sum_{j=0}^{m-1} |C_j|$ the written incidence count, and $q = |A|$. The numeric value of n need not be bounded by L , but m , r , and q are each $O(L)$. The forward program parses and sorts at most r written elements, then tests at most $qm = O(L^2)$ active-element/member pairs. Integer parsing, comparison, hashing, and output conversion act on at most L decimal digits per source integer. Even with hash collisions, these operations have polynomial bit cost; $O(L^4)$ is a coarse bound for the implemented construction.

The target contains m universe elements, q indexed sets, and exactly r incidences. Its indices have $O(\log(L + 1))$ digits, so its JSON length is $O((m + q + r) \log(L + 1))$. The algorithm never loops over

the numeric value of n . It disables the Python runtime’s default decimal-digit conversion limit so that every finite legal integer can be parsed, subject to available memory.

Let T be the target-output length. The decoder rebuilds A from the written source members and maps at most q selected indices back to source elements. Its bit time is polynomial in $L + T$, with the same coarse fourth-power bound. Its output has at most q source integers of at most L digits, giving length $O(L^2 + T)$. These are worst-case bounds on the maps, not claims about the running time of a set-cover solver.

6. Conclusion

The reviewed rule supplies executable forward construction and optimal-output recovery for the complete stated optimization domains. Active-element indexing makes the classical incidence duality polynomial under a compact decimal universe-size encoding. The incidence theorem is known [1, 2]; this result contributes a full-domain implementation and an encoding-aware proof. It makes no claim of a faster target solver or a new combinatorial theorem.

References

- [1] Marek Cygan, Holger Dell, Daniel Lokshtanov, Dániel Marx, Jesper Nederlof, Yoshio Okamoto, Ramamohan Paturi, Saket Saurabh, and Magnus Wahlström. “On Problems as Hard as CNF-SAT.” 2015 version, Section 4.1, paragraph preceding Observation 4.1 and Observation 4.1. <https://sites.cs.ucsb.edu/~daniello/papers/problemsAsHardAsSatJ.pdf>
- [2] CodingThrust/problem-reductions. “[Rule] MinimumHittingSet to MinimumSetCovering.” Issue 1094, opened 1 August 2026, “Reduction Algorithm” and “Size Overhead” sections. <https://github.com/CodingThrust/problem-reductions/issues/1094>

A. Verification and reproducibility

The 120 fixed prepared cases include 12 hand-designed edge cases and 108 seeded random cases, with source universe sizes at most seven and family sizes at most eight. Z3 4.16.0.0 computes their optima; independent exhaustive enumeration agrees on each source label. Executing the actual target solver and decoder checks 194 optimal target outputs. A separate enumerator covers all 689 indexed set systems with source universe size at most three and family length at most three, checking 821 optimal target outputs. Sparse regressions exercise four sources, including 5,001-digit universe sizes, empty families and members, and high-valued selected source indices. The independent reviewer added a separate three-source, four-output sparse check with alternate optimal recoveries. These are finite checks; the theorem establishes the unbounded claim.

The repository uses Python 3.12.14, uv 0.12.17, locked Z3 Python 4.16.0.0, and Typst 0.15.1. From the repository root, run:

```
uv sync --locked
uv run python campaigns/minimum-hitting-set-set-cover/work/check.py --self-test
uv run python campaigns/minimum-hitting-set-set-cover/work/check.py --candidate
    campaigns/minimum-hitting-set-set-cover/work/algorithm.py
uv run python campaigns/minimum-hitting-set-set-cover/work/verify.py --candidate
    campaigns/minimum-hitting-set-set-cover/work/algorithm.py
uv run python campaigns/minimum-hitting-set-set-cover/rounds/002/large_sparse.py
uv run python campaigns/minimum-hitting-set-set-cover/reviews/followup/
    check_sparse.py
```

The first candidate command invokes the forward CLI with a source JSON object. The checker then invokes the extraction CLI with `{"source": ..., "target_solution": ...}`. For direct use, the two modes are `uv run python campaigns/minimum-hitting-set-set-cover/work/algorithm.py` and `uv run python campaigns/minimum-hitting-set-set-cover/work/algorithm.py --extract`. The fixed corpus, generator, oracle encoding, command contract, proof, verification report, and independent reviews are retained in the campaign repository. No randomness occurs during construction or recovery.